

Simulazione Boids con Predatori

Architettura, algoritmi e guida all'uso

Progetto C++17 — SFML 2.x

Questo documento descrive l'architettura del simulatore, le regole di volo implementate, le scelte di refactoring rispetto alla versione originale e le istruzioni per compilare ed eseguire il progetto.

1. Introduzione

Il modello **Boids** (da *bird-oid objects*), introdotto da Craig Reynolds nel 1987, simula il comportamento collettivo emergente di uno stormo applicando tre semplici regole locali a ciascun agente:

- **Coesione** — il boid si muove verso il centro di massa dei vicini.
- **Allineamento** — il boid allinea la propria velocità a quella media dei vicini.
- **Separazione** — il boid mantiene una distanza minima dai vicini più prossimi.

Questo progetto estende il modello base con:

- Più stormi distinti (ID 0, 1, 2) che si separano tra loro.
- Un **predatore** (ID 999) che insegue le prede e da cui i boid fuggono con forza maggiore.
- **Ostacoli** posizionabili interattivamente con il mouse.
- Repulsione morbida dai bordi della finestra.
- Aggiornamento **veramente simultaneo**: tutti i delta-v vengono calcolati prima di aggiornare le posizioni.

L'implementazione è in **C++17** con la libreria grafica **SFML 2.x**.

2. Struttura del Progetto

File	Contenuto
PredatorBoids.hpp	Dichiarazioni: classe Boid, Ostacolo, funzioni libere, costanti
PredatorBoids.cpp	Implementazioni: regole di volo, aggiornamento simultaneo
main.cpp	Loop principale, gestione finestra e input, parametri
boid.test.cpp	Suite di unit test (framework doctest, header-only)

3. Architettura del Codice

3.1 Classe Boid

La classe **Boid** incapsula posizione (`pos_`), velocità (`vel_`) e identificatore di stormo (`idStormo_`). L'interfaccia pubblica espone solo metodi `const` per la lettura e metodi con effetti espliciti:

Metodo	Descrizione
<code>calcolaDeltaV(...)</code> <code>const</code>	Calcola il delta-v senza modificare lo stato (read-only)
<code>applica(dv)</code>	Applica dv e avanza la posizione: <code>vel_ += dv</code> ; <code>pos_ += vel_</code>
<code>controllaV(...)</code>	Clamp velocità entro <code>[minV, maxV]</code> (o <code>maxVPred</code> per predatori)
<code>bordi(...)</code>	Repulsione soft proporzionale alla profondità nel margine
<code>disegno(...)</code>	Rendering SFML del boid come cerchio colorato

I predatori (`idStormo_ == 999`) sono distinti tramite la costante **ID_PREDATORE** nell'header, eliminando numeri magici nel codice.

3.2 Regole di Volo (funzioni libere)

Le quattro funzioni di steering sono implementate come **funzioni libere** nel namespace `brd`, separate dalla classe. Questo le rende testabili in isolamento e facili da estendere:

Funzione	Raggio di vista	Logica
<code>coesione()</code>	RAGGIO_VISTA_STORMO (30 px)	Centro di massa → steering
<code>allineamento()</code>	RAGGIO_VISTA_STORMO (30 px)	Velocità media → steering
<code>separazione()</code>	SOGLIA_SEPARAZIONE (25 px)	diff/d ponderata per fattore
<code>evitareOstacoli()</code>	RAGGIO_OSTACOLO (50 px)	Repulsione $\propto 1/d$

Tutti i raggi sono definiti come inline `constexpr double` nell'header: modificabili in un unico punto senza toccare le implementazioni.

3.3 Aggiornamento Simultaneo

La funzione libera **aggiorna()** risolve il problema centrale della versione originale: i boid venivano aggiornati sequenzialmente, quindi ogni boid influenzava il calcolo dei successivi *nello stesso tick*, producendo comportamenti dipendenti dall'ordine in cui erano inseriti nel vettore.

L'aggiornamento corretto avviene in due fasi distinte:

- **Fase 1 — Lettura (read-only):** `std::transform` chiama `calcolaDeltaV` su ciascun boid usando la snapshot *corrente* dello stormo. Essendo il metodo `const`, è garantito che nessuno stato venga modificato.
- **Fase 2 — Scrittura:** un secondo ciclo applica tutti i delta-v con `applica()`. Solo a questo punto posizioni e velocità cambiano.

Questo schema è concettualmente analogo al *double buffering* nei sistemi real-time e ai metodi *predictor-corrector* nell'integrazione numerica.

3.4 Logica del Predatore

I boid con `ID == ID_PREDATORE` seguono regole distinte:

- Inseguono le prede nel raggio `RAGGIO_VISTA_PRED` (40 px) tramite `coesione` e `allineamento`.
- La separazione viene applicata rispetto agli altri predatori per evitare sovrapposizioni.
- La velocità massima è limitata a `MAX_V_PRED` (0.25 px/tick), inferiore a quella dei boid normali (0.5 px/tick): il predatore è lento ma persistente.
- I boid normali che vedono un predatore nel loro `RAGGIO_VISTA_ALTRO` (50 px) fuggono con peso `PESO_PREDATORE`, significativamente maggiore di `PESO_SEPARAZIONE`.
- Il predatore è disegnato in rosso con raggio doppio (5 px vs 3 px).

4. Miglioramenti Rispetto alla Versione Originale

Problema originale	Soluzione adottata
Aggiornamento sequenziale: ogni boid influenzava i successivi nello stesso tick	calcolaDeltaV (const) + applica(): aggiornamento veramente simultaneo
evitareOstacoli: nessun return se nessun ostacolo nel raggio → undefined behavior	Return esplicito {0,0} aggiunto alla fine
separazione: parametri float, resto usa double → conversioni implicite	Uniformati a double in tutta la codebase
std::rand() nel costruttore default, non seedato	Solo std::mt19937 con random_device in main
bordi(): divisione per pos.x → instabilità numerica vicino a pos.x ≈ 0	Repulsione proporzionale alla profondità di penetrazione nel margine
controlloV(): velocità minima sommata come vettore fisso → direzione alterata	Velocità minima mantenendo la direzione: $vel/n * minV$
Numeri magici sparsi (999, 30, 50, 40, 25)	Costanti constexpr con nome nell'header (ID_PREDATORE, RAGGIO_VISTA_STORMO, ...)
Parametri hardcoded in mezzo al main	constexpr statici con commenti in cima a main.cpp
Test con API obsoleta (SetPos, SetVel, SetId, metodi membro)	Riscritti con API corrente (getPos, funzioni libere)
Casi limite non testati	Aggiunti test per lista vuota, ostacolo fuori raggio, separazione fuori soglia
Commenti di debug e codice commentato nell'header	Header pulito, solo dichiarazioni e documentazione
Click destro non gestito	Click destro rimuove l'ultimo ostacolo
Nessun framerate limit → CPU al 100%	window.setFramerateLimit(60) nel main

5. Parametri della Simulazione

Tutti i parametri sono centralizzati in due punti:

- **PredatorBoids.hpp**: costanti fisiche (raggi di vista, soglie).
- **main.cpp**: parametri della simulazione (N boid, pesi, velocità).

Costante / Parametro	Valore default	Descrizione
RAGGIO_VISTA_STORMO	30 px	Raggio coesione/allineamento (stesso stormo)
RAGGIO_VISTA_ALTRO	50 px	Raggio visione boid di un altro stormo
RAGGIO_VISTA_PRED	40 px	Raggio di caccia del predatore
SOGLIA_SEPARAZIONE	25 px	Distanza sotto cui scatta la repulsione
RAGGIO_OSTACOLO	50 px	Raggio di evitamento degli ostacoli
N_BOIDS	300	Numero totale di boid (predatori inclusi)

Costante / Parametro	Valore default	Descrizione
FREQ_PREDATORI	10	Un predatore ogni N boid ($300/10 = 30$ predatori)
PESO_COESIONE	0.001	Steering verso il centro di massa
PESO_ALLINEAMENTO	0.1	Steering verso la velocità media
PESO_SEPARAZIONE	0.4	Repulsione generica tra boid vicini
PESO_PREDATORE	1.0	Peso amplificato fuga dal predatore
PESO_OSTACOLI	0.09	Repulsione dagli ostacoli
MAX_VELOCITY	0.5 px/tick	Velocità massima boid normali
MAX_V_PRED	0.25 px/tick	Velocità massima predatore
MIN_VELOCITY	0.08 px/tick	Velocità minima (garantisce moto continuo)
REPULSIONE_BORDI	1.5	Forza di repulsione dai bordi della finestra

6. Unit Test

I test sono implementati con il framework **doctest** (header-only) nel file `boid.test.cpp`. Coprono tutti i comportamenti critici del sistema:

Test case	Verifica
Separazione – stesso stormo	Repulsione nella direzione corretta, peso <code>pesoSeparazione</code>
Separazione – fuga dal predatore	Peso <code>pesoPredatore</code> applicato quando l'altro boid è <code>ID_PREDATORE</code>
Separazione – fuori soglia	Nessuna repulsione se $d > SOGLIA_SEPARAZIONE$
Allineamento – media velocità	Steering verso la velocità media dei vicini
Allineamento – lista vuota	Ritorna (0,0) senza crash
Coesione – centro di massa	Steering verso il baricentro calcolato correttamente
Coesione – lista vuota	Ritorna (0,0) senza crash
EvitareOstacoli – vicino	Repulsione inversamente proporzionale alla distanza
EvitareOstacoli – fuori raggio	Nessuna forza se $d > RAGGIO_OSTACOLO$
EvitareOstacoli – lista vuota	Ritorna (0,0) senza crash
ControlloV – clamping massimo	Norma della velocità $\leq \text{maxV}$ dopo il clamping
ControlloV – minimo garantito	Norma della velocità $\geq \text{minV}$
ControlloV – predatore	Norma $\leq \text{maxVPred}$ per boid con <code>ID_PREDATORE</code>
aggiorna – posizione avanza	La posizione cambia dopo un tick di simulazione

7. Compilazione e Utilizzo

7.1 Dipendenze

- Compilatore C++17: GCC 10+ o Clang 11+
- SFML 2.5+ (libsFML-dev su Debian/Ubuntu, brew install sfml su macOS)
- doctest.h (header-only) nella directory dei sorgenti per i test — github.com/doctest/doctest

7.2 Compilazione con g++

Simulazione principale:

```
g++ -std=c++17 -O2 -Wall -Wextra \
main.cpp PredatorBoids.cpp \
-lsfml-graphics -lsfml-window -lsfml-system \
-o boids
```

Suite di test:

```
g++ -std=c++17 -O2 -Wall -Wextra \
boid.test.cpp PredatorBoids.cpp \
-lsfml-graphics -lsfml-window -lsfml-system \
-o boids_test && ./boids_test
```

7.3 CMakeLists.txt (opzionale)

```
cmake_minimum_required(VERSION 3.16)
project(Boids CXX)
set(CMAKE_CXX_STANDARD 17)
find_package(SFML 2.5 COMPONENTS graphics window system REQUIRED)
add_executable(boids main.cpp PredatorBoids.cpp)
target_link_libraries(boids sfml-graphics sfml-window sfml-system)
add_executable(boids_test boid.test.cpp PredatorBoids.cpp)
target_link_libraries(boids_test sfml-graphics sfml-window sfml-system)
enable_testing()
add_test(NAME UnitTest COMMAND boids_test)
```

7.4 Controlli a Runtime

Input	Azione
Click sinistro	Aggiunge un ostacolo nella posizione del cursore
Click destro	Rimuove l'ultimo ostacolo inserito
Chiudi finestra	Termina la simulazione

7.5 Personalizzazione

Per modificare il comportamento della simulazione è sufficiente cambiare le costanti in cima a main.cpp (N_BOIDS, pesi, velocità) e le costanti fisiche in PredatorBoids.hpp (raggi di vista, soglie). Non è necessario toccare le implementazioni.

8. Riferimenti

- Reynolds, C.W. (1987). *Flocks, Herds, and Schools: A Distributed Behavioral Model*. SIGGRAPH Computer Graphics, 21(4), 25–34. red3d.com/cwr/papers/1987/boids.html

- SFML Documentation 2.6: sfml-dev.org/documentation/2.6/
- doctest — C++11/17 single-header testing framework: github.com/doctest/doctest
- ISO/IEC 14882:2017 — Programming Language C++ (C++17 Standard)